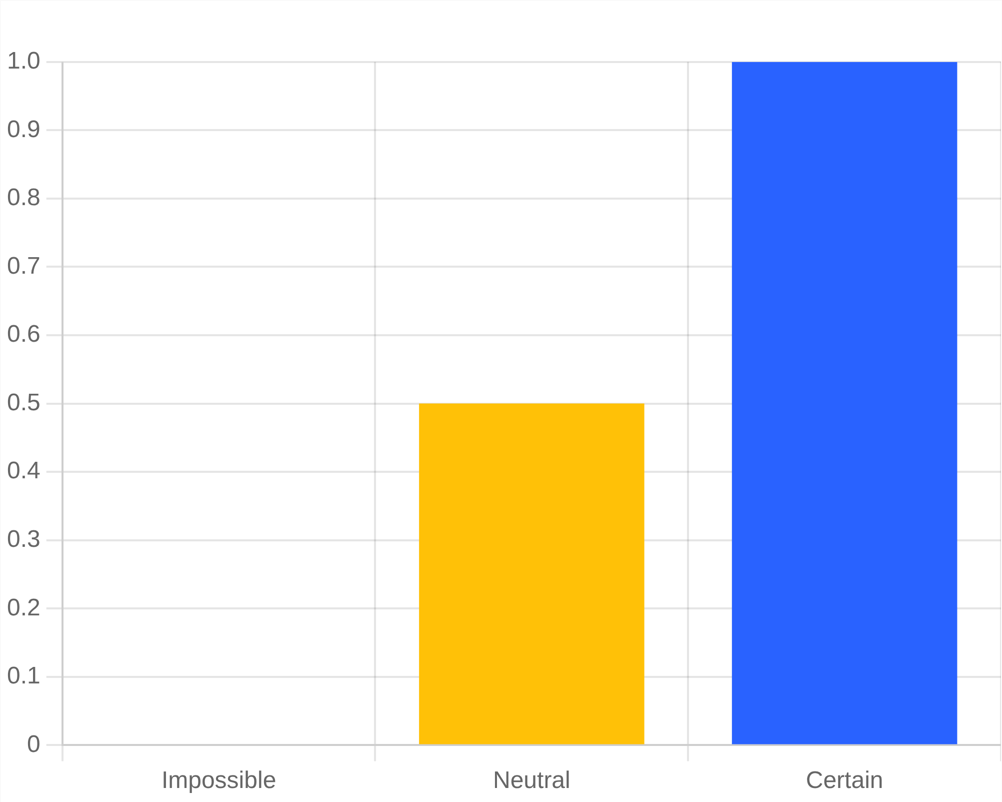
PROBABILITY IN PYTHON

From Concepts to Code

MEASURING THE LIKELY

- Probability quantifies how likely an event is to occur.
- Values range from 0 (Impossible) to 1 (Certain).
- 0.5 represents a balanced 50% chance.
- Coin Toss Example: P(Head) = 1/2 = 0.5.



THE PROBABILITY SCALE

WHY IT MATTERS

In real life, few things are certain. Probability provides a way to make **informed decisions** despite this.

It is the backbone of **Artificial Intelligence**, allowing machines to learn from data and make predictions.

WEATHER

FINANCE

HEALTHCARE

AI / ML

STRATEGIC GOALS

WHAT PROBABILITY ACHIEVES

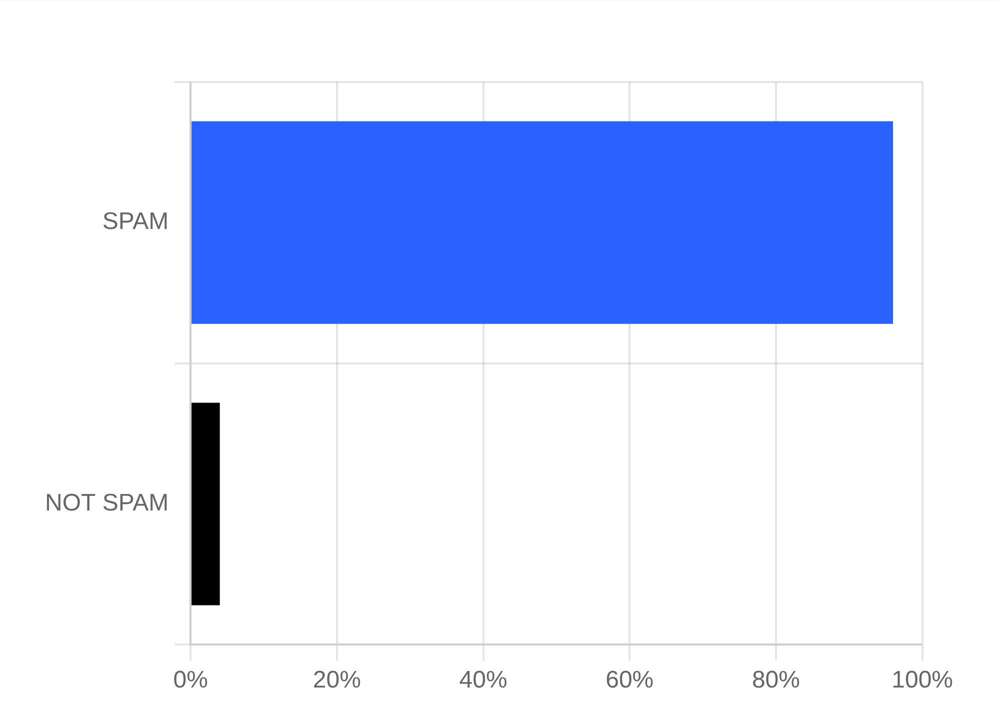
- **QUANTIFYING RISK**
  Measure and analyze uncertainty in complex systems.
- **PREDICTIVE POWER**
  Analyze past data to forecast future outcomes.
- **DECISION SUPPORT**
  Provide a logical basis for action under uncertainty.
- **AI FOUNDATION**
  Build models that recognize patterns and classify data.

INDUSTRY APPLICATIONS

DOMAIN	USE OF PROBABILITY
WEATHER	Predicting precipitation and extreme weather events.
BANKING	Credit scoring, fraud detection, and market risk analysis.
HEALTHCARE	Diagnostic tools and patient outcome predictions.
E-COMMERCE	Personalized product recommendations and demand forecasting.
SOCIAL MEDIA	Algorithmic content curation and targeted advertising.
SELF-DRIVING CARS	Real-time object detection and path planning.

AI SPAM FILTERING

- AI models assign probabilities to incoming emails.
- Example: A suspicious email arrives.
- The model calculates likelihood for each class.
- Highest probability determines the final action.



RESULT: CLASSIFIED AS SPAM

CODING BASICS

- Standard Python's random module is ideal for basic logic.
- Simple functions like choice() and randint() handle basic events.
- Law of Large Numbers: Repeat experiments to estimate true probability.

COIN TOSS SIMULATION

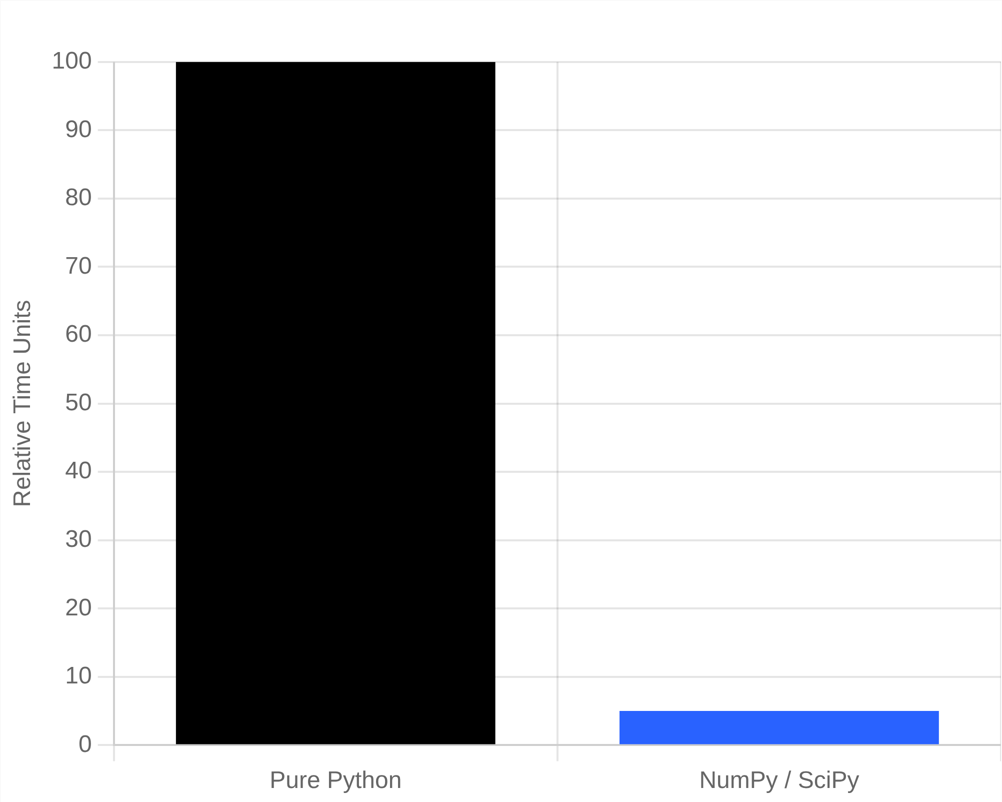
```
import random
result = random.choice(["Head", "Tail"])
```

DICE ROLL SIMULATION

```
import random
dice = random.randint(1, 6)
```

SCALING LIMITS

- **Performance Bottlenecks:** Significantly slower for large datasets.
- **Manual Effort:** Requires verbose loops for repetitive simulations.
- **Memory Inefficiency:** Not optimized for millions of records.
- **AI/ML Constraints:** Lacks advanced mathematical optimizations.



EXECUTION TIME COMPARISON

NUMPY POWER

- **Vectorization:** Operations on entire arrays at once, no slow loops.
- **High Speed:** Optimized C-based implementation for large-scale computing.
- **Weighted Events:** Easily simulate events with unequal probabilities.

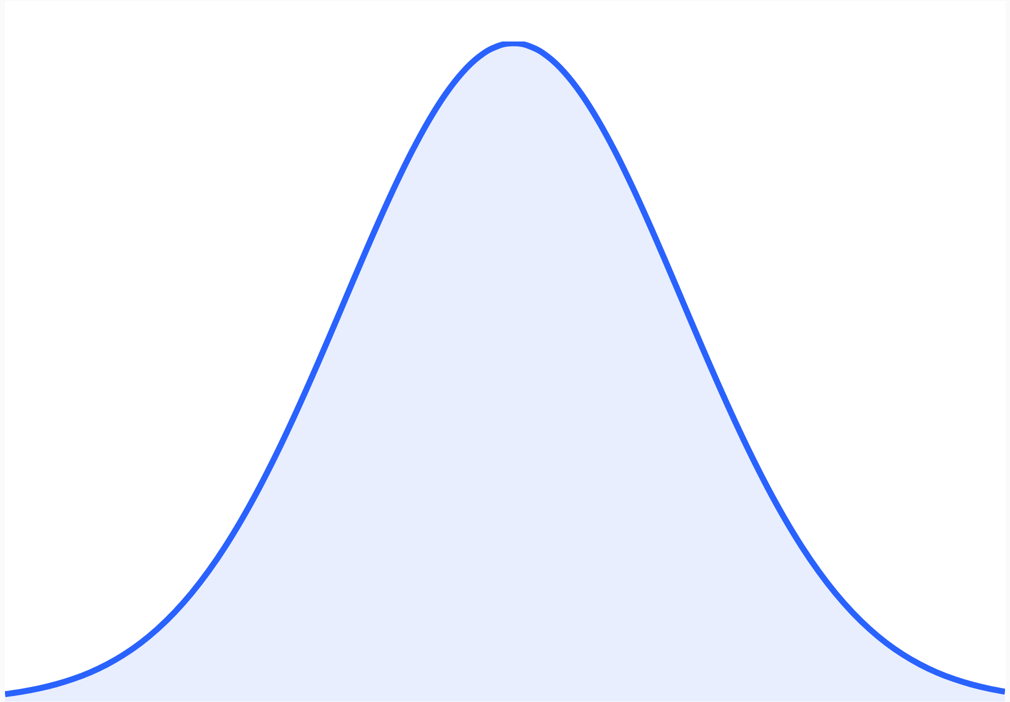
WEIGHTED DISTRIBUTION SIMULATION

```
import numpy as np

# 70% Head, 30% Tail
results = np.random.choice(
  ["Head", "Tail"],
  size=100,
  p=[0.7, 0.3]
)
```

ADVANCED ANALYTICS

- **SciPy** provides specialized functions for advanced probability distributions.
- Calculate **CDF** for statistical modeling and hypothesis testing.
- Core Uses: Confidence Intervals, Statistical Testing, and Scientific Computations.



NORMAL DISTRIBUTION CURVE

CHOOSING THE RIGHT TOOL

FEATURE	NORMAL PYTHON	NUMPY	SCIPY
BEST FOR	Learning & Basic Logic	Fast Numerical Computing	Advanced Statistics
RANDOMNESS	random module	numpy.random	Statistical distributions
SPEED	Slower (Loops)	**Very Fast (Vectorized)**	Fast & Specialized
LARGE DATA	Not Efficient	**Highly Efficient**	Highly Efficient
AI/ML ROLE	Conceptual Prototyping	Data Preprocessing	Modeling & Testing

MODERN INTELLIGENCE

THE PROBABILISTIC FUTURE

- Probability turns **uncertainty** into actionable data for AI models.
- **NumPy** is the essential workhorse for fast, large-scale simulations.
- **SciPy** provides the mathematical depth for advanced statistical analysis.

FINAL INSIGHT

Machine Learning models don't "guess"—they calculate probabilities to choose the most likely outcome. This foundation is what makes modern AI reliable and predictive.